PRATİK REHBER

Ansible ile Ağ Otomasyonuna Giriş

Cisco & Juniper Ortamlarında Pratik Network Automation

Ağ Yöneticileri için Kapsamlı Başlangıç Kılavuzu

2025

İçindekiler

Bölüm 1: Ağ Otomasyonuna Giriş

1.1 Neden Ağ Otomasyonu?

1.2 Ansible Nedir?

1.3 Ansible'ın Ağ Cihazlarında Çalışma Prensibi

1.4 Ansible Mimarisi

Bölüm 2: Kurulum ve Ortam Hazırlığı

2.1 Sistem Gereksinimleri

2.2 Ansible Kurulumu

2.3 SSH ve Erişim Ayarları

2.4 Proje Dizin Yapısı

Bölüm 3: Inventory Dosyası Yönetimi

3.1 Inventory Dosyası Nedir?

3.2 INI Formatında Inventory

3.3 YAML Formatında Inventory

3.4 Değişkenler ve group_vars

Bölüm 4: Ansible Ağ Modülleri

4.1 Bağlantı Yöntemleri

4.2 ios_command ve ios_config

4.3 junos_command ve junos_config

4.4 Modül Seçim Rehberi

Bölüm 5: Playbook #1 - Toplu VLAN Oluşturma

5.1 Senaryo Tanımı

5.2 Cisco IOS VLAN Yönetimi

5.3 Juniper JunOS VLAN Yönetimi

5.4 Adım Adım Açıklama

Bölüm 6: Playbook #2 - Konfigürasyon Yedekleme

6.1 Senaryo Tanımı

6.2 Cisco Konfigürasyon Yedekleme

6.3 Juniper Konfigürasyon Yedekleme

6.4 Otomatik Zamanlamalı Yedekleme

Bölüm 7: Playbook #3 - Arayüz Yapılandırması

7.1 Senaryo Tanımı

7.2 Cisco Arayüz Yapılandırması

7.3 Juniper Arayüz Yapılandırması

7.4 Jinja2 Şablonları ile Dinamik Yapılandırma

Bölüm 8: İleri Seviye ve Best Practices

8.1 Hata Yakalama ve Retry

8.2 Vault ile Şifreli Değişkenler

8.3 Idempotency ve Dry-Run

8.4 Performans Optimizasyonu

8.5 Dokümantasyon ve Versiyon Kontrolü

Ekler

Ek A: Hızlı Başvuru Kartı

Ek B: Sık Karşılaşılan Hatalar

Ek C: Faydalı Kaynaklar

Bölüm 1: Ağ Otomasyonuna Giriş

1.1 Neden Ağ Otomasyonu?

Modern ağ altyapıları, yüzlerce hatta binlerce cihazdan oluşan karmaşık yapılardır. Geleneksel yönetim yaklaşımında, ağ yöneticileri her cihaza tek tek bağlanarak yapılandırma değişiklikleri gerçekleştirir. Bu yöntem hem zaman alıcı hem de hata riski yüksektir.

Manuel yönetimin temel sorunları:

- **Zaman maliyeti:** 50 switch'e tek tek VLAN tanımlamak saatler alabilir
- **İnsan hatası:** Komutun yanlış yazılması, parametrenin atlanması gibi hatalar kaçınılmazdır
- **Tutarsızlık:** Farklı yöneticiler farklı yaklaşımlar kullanabilir
- **Ölçeklenebilirlik:** Cihaz sayısı arttıkça yönetim imkansızlaşır
- **Rollback zorluğu:** Hatalı değişikliği geri almak manuel olarak çok zordur

Ağ otomasyonu, tüm bu sorunlara sistematik çözümler sunar. Yapılandırmalar kod olarak tanımlanır, test edilir ve tutarlı bir şekilde uygulanır. Değişiklikler versiyon kontrol sistemiyle izlenir ve gerektiğinde saniyeler içinde geri alınabilir.

> **Bilgi: Infrastructure as Code (IaC)**
>
> Ağ otomasyonu, Infrastructure as Code (IaC) felsefesinin bir uzantısıdır. Bu yaklaşımda altyapı yapılandırmaları, insanların okuyup anlayabileceği dosyalarda (YAML, JSON vb.) tanımlanır ve otomasyon araçları aracılığıyla uygulanır.

1.2 Ansible Nedir?

Ansible, Red Hat tarafından geliştirilen açık kaynaklı bir otomasyon aracıdır. Sunucu yapılandırma, uygulama dağıtımı ve ağ otomasyonu gibi birçok alanda kullanılır. Ansible'ın popülerliğinin temel nedenleri şunlardır:

Özellik	Açıklama	Ağ Otomasyonu İçin Önemi

Agentless	Cihazlara ek yazılım kurulumu gerektirmez	Ağ cihazlarına yazılım yüklemek riskli ve karmaşıktır; Ansible SSH/NETCONF ile çalışır
Idempotent	Aynı işlem tekrar çalıştırıldığında durumu değiştirmez	VLAN zaten varsa tekrar oluşturmaz; güvenli ve öngörülebilir
YAML Tabanlı	Playbook'lar YAML formatında yazılır	Okunması ve yazılması kolaydır; teknik ekip kolayca adapte olur
Geniş Modül Desteği	Cisco, Juniper, Arista, MikroTik vb. için hazır modüller	Her üretici için özel modüller mevcuttur
Ücretsiz	Açık kaynak; AWX/Ansible Tower ile kurumsal destek	Bütçe dostu; kurumsal ihtiyaçlarda Tower seçeneği

Ansible, **kontrol düğümü (control node)** üzerinden çalışır. Kontrol düğümü, yöneticinin bilgisayarı veya ayrı bir sunucu olabilir. Playbook'lar bu düğümde çalıştırılır ve hedef cihazlara SSH, NETCONF veya API üzerinden erişilir.

1.3 Ansible'ın Ağ Cihazlarında Çalışma Prensibi

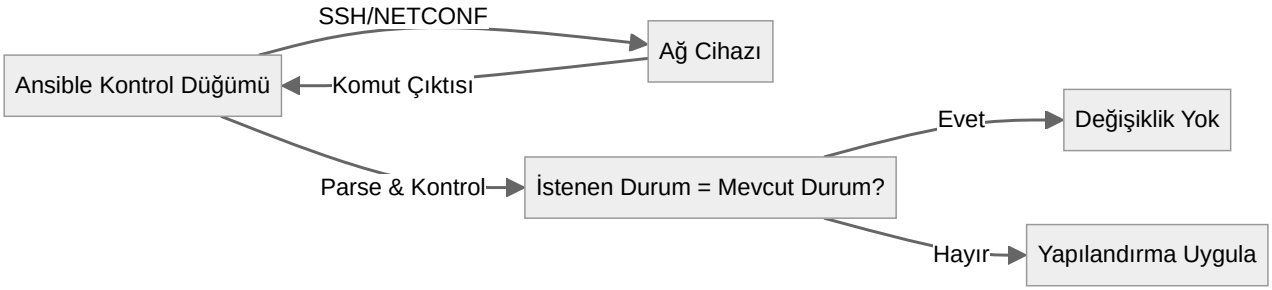
Ansible, sunuculardan farklı olarak ağ cihazlarında çalışırken farklı bağlantı yöntemleri kullanır. Bu bağlantı yöntemleri, cihazın desteklediği protokollere göre seçilir.

Temel Bağlantı Yöntemleri

Bağlantı Yöntemi	Kullanım Alanı	Protokol	Avantaj
`network_cli`	Cisco IOS/IOS-XE, NX-OS, JunOS, EOS	SSH	Komut satırı tabanlı; en yaygın yöntem
`netconf`	Cisco IOS-XR, JunOS	SSH üzerinden NETCONF	Yapılandırma doğrulama ve rollback desteği
`httpapi`	Cisco NX-API, RESTCONF destekleyen cihazlar	HTTP/HTTPS	Modern API tabanlı; JSON desteği

`local`	Eski yöntem; önerilmez	SSH (paramiko)	Geriye dönük uyumluluk

Ansible'ın ağ cihazıyla etkileşimi tipik olarak şu adımları izler:



> **Önemli: Ağ Cihazlarında Python Yoktur**
>
> Sunucu otomasyonunda Ansible, hedef sunucuya Python kodu kopyalar ve çalıştırır. Ancak ağ cihazlarında (çoğu switch/router) Python çalıştırma ortamı yoktur. Bu nedenle Ansible, tüm işlemleri **kontrol düğümünde** gerçekleştirir ve cihazla sadece SSH/API üzerinden komut alışverişi yapar.

1.4 Ansible Mimarisi

Ansible'ın çalışma mimarisini anlamak, playbook'ları daha etkili yazmak için temeldir. Temel bileşenler şunlardır:

Inventory
: Yönetilecek cihazların listesini ve özelliklerini içeren dosya. IP adresleri, hostnameler, grup bilgileri ve cihaza özel değişkenler burada tanımlanır.

Modules
: Ansible'ın cihaz üzerinde gerçekleştirdiği işlemleri tanımlayan Python kodlarıdır. `ios_config`, `junos_command` gibi ağ modülleri, üretici özel komutları çalıştırır.

Playbook
: YAML formatında yazılan otomasyon senaryolarıdır. Hangi cihazlarda (hosts), hangi modüllerle (tasks) ne yapılacağını tanımlar.

Roles
: Tekrar kullanılabilir playbook parçalarıdır. VLAN yönetimi, yedekleme gibi görevleri modüler hale getirir.

Facts

Ansible'ın cihazdan otomatik topladığı bilgilerdir. Cihaz modeli, IOS versiyonu, arayüz listesi gibi bilgiler facts olarak erişilebilir.

Variables

Playbook'larda dinamiklik sağlayan değerlerdir. Inventory'de, playbook'da veya ayrı dosyalarda tanımlanabilir.

> **Ansible Çalışma Akışı Özeti**
>
> 1. Kontrol düğümünde `ansible-playbook` komutu çalıştırılır
> 2. Inventory dosyası okunur; hedef cihazlar belirlenir
> 3. Her cihaz için SSH/NETCONF bağlantısı kurulur
> 4. Modüller sırayla çalıştırılır; cihazdan çıktı alınır
> 5. Çıktılar parse edilir; istenen durum karşılaştırılır
> 6. Sonuçlar özet olarak ekrana yazdırılır

Bölüm 2: Kurulum ve Ortam Hazırlığı

2.1 Sistem Gereksinimleri

Ansible'ı çalıştırmak için ihtiyaç duyulan kaynaklar oldukça mütevazıdır. Ağ otomasyonu senaryolarında, kontrol düğümü ortalama bir laptop bile olabilir.

Bileşen	Minimum	Önerilen
İşletim Sistemi	Linux (herhangi bir dağıtım)	Ubuntu 22.04/24.04 LTS, RHEL 9, Debian 12
Python	3.9	3.11 veya üzeri
RAM	512 MB	2 GB
Disk	500 MB	5 GB (log ve yedekler için)
Ağ Erişimi	Hedef cihazlara SSH erişimi	Management VLAN'a erişim, 22/830 portları açık

> **Uyarı: Windows Kontrol Düğümü**
>
> Ansible resmi olarak Windows üzerinde kontrol düğümü olarak çalışmaz. Windows kullanıcılarının WSL2 (Windows Subsystem for Linux) kurmaları veya bir Linux sanal makine kullanmaları önerilir.

2.2 Ansible Kurulumu

Adım 1: Paket Yöneticisi ile Kurulum

En kolay kurulum yöntemi, işletim sisteminin paket yöneticisini kullanmaktır:

```
Ubuntu/Debian
```

```
# Paket listesini güncelle
sudo apt update

# Gerekli bağımlılıkları yükle
sudo apt install -y software-properties-common

# Ansible PPA ekle (daha güncel sürüm için)
sudo add-apt-repository --yes --update ppa:ansible/ansible

# Ansible'ı yükle
sudo apt install -y ansible

# Kurulumu doğrula
ansible --version
```

RHEL/CentOS/Rocky Linux

```
# EPEL repository etkinleştir
sudo dnf install -y epel-release

# Ansible'ı yükle
sudo dnf install -y ansible

# Kurulumu doğrula
ansible --version
```

Adım 2: pip ile Kurulum (Alternatif)

En güncel sürümü yüklemek veya sanal ortam kullanmak isterseniz pip ile kurulum yapabilirsiniz:

```
# Python sanal ortam oluştur
python3 -m venv ~/ansible-venv

# Sanal ortamı etkinleştir
source ~/ansible-venv/bin/activate

# Ansible ve ağ modüllerini yükle
pip install ansible ansible-pylibssh paramiko

# Kurulumu doğrula
ansible --version
```

Adım 3: Ağ Modüllerini Doğrulama

Ansible ile birlikte Cisco ve Juniper modülleri varsayılan olarak gelir. Modüllerin yüklü olduğunu doğrulayın:

```
# Cisco modüllerini listele
ansible-doc -l | grep ios_

# Juniper modüllerini listele
ansible-doc -l | grep junos_

# Belirli bir modülün dokümantasyonunu görüntüle
ansible-doc ios_config
```

Beklenen Çıktı Örneği:

```
ios_banner        Cisco IOS banner yapılandırması
ios_command       Cisco IOS komut çalıştırma
ios_config        Cisco IOS yapılandırma yönetimi
ios_facts         Cisco IOS cihaz bilgisi toplama
ios_vlan          Cisco IOS VLAN yönetimi
ios_interface     Cisco IOS arayüz yönetimi
```

2.3 SSH ve Erişim Ayarları

Ansible'ın ağ cihazlarına bağlanabilmesi için SSH erişimi gerekir. Güvenli ve sorunsuz bir otomasyon ortamı için aşağıdaki adımları izleyin.

Cisco IOS Cihazlarında SSH Hazırlığı

Cisco switch veya router'ınızda aşağıdaki temel yapılandırmanın olduğundan emin olun:

```
! Management VLAN arayüzü
interface Vlan10
 ip address 192.168.10.10 255.255.255.0
 no shutdown

! SSH erişimi için domain adı
ip domain-name network.local

! SSH anahtar çifti oluşturma
crypto key generate rsa modulus 2048

! SSH sürüm 2 zorunlu
ip ssh version 2

! Kullanıcı oluşturma (privilege 15 = tam yetki)
username ansible privilege 15 secret Sifre123!

! VTY hatlarında SSH etkinleştirme
line vty 0 15
 transport input ssh
 login local

! Enable şifresi (gerekirse)
enable secret EnableSifre456!
```

Kod Açıklaması

- `interface Vlan10` : Management VLAN arayüzü; IP adresi atanır
- `crypto key generate rsa` : SSH bağlantısı için gerekli RSA anahtar çifti oluşturur
- `username ansible privilege 15` : Otomasyon için özel kullanıcı; seviye 15 = enable moduna gerek kalmadan tam yetki
- `transport input ssh` : Sadece SSH erişimine izin verir (Telnet'i devre dışı bırakır)

Juniper JunOS Cihazlarında SSH Hazırlığı

```
# Management arayüzü yapılandırma
set interfaces me0 unit 0 family inet address 192.168.10.20/24

# SSH hizmetini etkinleştirme
set system services ssh

# Kullanıcı oluşturma (super-user sınıfı = tam yetki)
set system login user ansible class super-user authentication plain-text-
password
# Şifre girmeniz istenecektir

# Root şifresi (SSH ile root login için gerekli olabilir)
set system root-authentication plain-text-password
```

Kontrol Düğümünde SSH Anahtar Yönetimi

Her seferinde şifre girmemek için SSH anahtar tabanlı kimlik doğrulama kullanabilirsiniz:

```
# SSH anahtar çifti oluştur (eğer yoksa)
ssh-keygen -t ed25519 -C "ansible@network"

# Cisco cihaza anahtarı kopyala (parola ile)
ssh-copy-id -o PreferredAuthentications=password ansible@192.168.10.10

# Juniper cihaza anahtarı manuel olarak ekleme
ssh ansible@192.168.10.20
# configure
#  set system login user ansible authentication ssh-ed25519 "PUBLIC KEY
BURAYA"
# commit
```

İpucu: SSH Config Dosyası

Birden fazla cihaz yönetiyorsanız, `~/.ssh/config` dosyası ile cihaz bazlı ayarlar tanımlayabilirsiniz:

```
Host sw-core-01
    HostName 192.168.10.10
    User ansible
    StrictHostKeyChecking no
    UserKnownHostsFile /dev/null
```

2.4 Proje Dizin Yapısı

Düzenli bir proje yapısı, playbook'ların yönetimini kolaylaştırır. Aşağıdaki dizin yapısını kullanmanız önerilir:

```
~/network-automation/
├── ansible.cfg           # Ansible yapılandırma dosyası
├── inventory/
│   ├── production/       # Üretim ortamı
│   │   ├── hosts.ini     # Üretim cihaz listesi
│   │   ├── group_vars/   # Grup bazlı değişkenler
│   │   │   ├── cisco.yml
│   │   │   └── juniper.yml
│   │   └── host_vars/    # Cihaz bazlı değişkenler
│   │       ├── sw-core-01.yml
│   │       └── sw-access-02.yml
│   └── staging/          # Test ortamı
│       └── hosts.ini
├── playbooks/
│   ├── vlan-management.yml
│   ├── config-backup.yml
│   └── interface-config.yml
├── templates/
│   ├── cisco_vlan.j2
│   └── juniper_interface.j2
├── files/
│   └── banner.txt
├── roles/                # İleride roller eklenebilir
└── outputs/              # Yedekler ve loglar
    └── backups/
```

ansible.cfg Yapılandırma Dosyası

`ansible.cfg` dosyası, Ansible'ın davranışını yapılandırmak için kullanılır. Ağ otomasyonu için önemli ayarlar:

```
[defaults]
inventory = inventory/production/hosts.ini
host_key_checking = False
timeout = 30
retry_files_enabled = False
interpreter_python = /usr/bin/python3

# Ağ modülleri için fork sayısını sınırla
forks = 10

# Loglama etkinleştir
log_path = outputs/ansible.log

[persistent_connection]
connect_timeout = 30
command_timeout = 30
connect_retry_timeout = 15
```

Kod Açıklaması

- `host_key_checking = False`: Yeni cihazlarda SSH host key kontrolünü devre dışı bırakır; lab/test ortamında kullanışlıdır
- `forks = 10`: Aynı anda 10 cihaza bağlanır; ağ cihazlarının CPU/bellek sınırlarını aşmamak için makul değer
- `connect_timeout = 30`: Yavaş cihazlarda bağlantı zaman aşımını önlemek için
- `log_path`: Tüm çalıştırmalar otomatik olarak loglanır; denetim ve sorun giderme için kritik

Bölüm 3: Inventory Dosyası Yönetimi

3.1 Inventory Dosyası Nedir?

Inventory (envanter) dosyası, Ansible'ın yöneteceği tüm cihazların listesini ve özelliklerini içeren temel dosyadır. Bu dosya olmadan Ansible hangi cihazlara bağlanacağını bilemez. Inventory dosyaları INI veya YAML formatında yazılabilir.

Inventory dosyası şu bilgileri içerir:

- **Host tanımları:** IP adresleri veya FQDN'ler
- **Grup tanımları:** Cihazları mantıksal gruplara ayırma (core, access, distribution)
- **Grup değişkenleri:** Bir grup cihaz için ortak değişkenler (os tipi, SSH kullanıcısı)
- **Host değişkenleri:** Tek bir cihaz için özel değişkenler

> **Inventory Hiyerarşisi**
>
> Ansible'da değişken önceliği (precedence) vardır. Aynı değişken birden fazla yerde tanımlanırsa, en spesifik olan kazanır: **host_vars > group_vars > inventory > playbook defaults**

3.2 INI Formatında Inventory

INI formatı, basit yapıları için en pratik yöntemdir. Köşeli parantezler içinde gruplar tanımlanır ve altına cihazlar yazılır.

`inventory/production/hosts.ini`

```
# ============================================
# ANSIBLE INVENTORY - ÜRETİM ORTAMI
# ============================================

# --- Core Switch'ler ---
[cisco_core_switches]
sw-core-01 ansible_host=192.168.10.10
sw-core-02 ansible_host=192.168.10.11

[juniper_core_switches]
jr-core-01 ansible_host=192.168.10.20

# --- Access Switch'ler ---
[cisco_access_switches]
sw-access-01 ansible_host=192.168.10.101
sw-access-02 ansible_host=192.168.10.102
sw-access-03 ansible_host=192.168.10.103

# --- Grup Birleştirmeleri ---
[cisco:children]
cisco_core_switches
cisco_access_switches

[juniper:children]
juniper_core_switches

[switches:children]
cisco
juniper

# --- Grup Değişkenleri ---
[cisco:vars]
ansible_network_os=ios
ansible_connection=network_cli
ansible_user=ansible
ansible_ssh_pass=Sifre123!
ansible_become=no
ansible_become_method=enable

[juniper:vars]
ansible_network_os=junos
ansible_connection=netconf
ansible_user=ansible
ansible_ssh_pass=JunSifre123!
ansible_port=830
```

Kod Açıklaması

- `[cisco_core_switches]` : Cihaz grubu; altındaki tüm cihazlar bu gruba aittir
- `ansible_host` : Cihazın IP adresi; isimden bağımsız olarak kullanılır
- `[cisco:children]` : Alt grup tanımı; cisco grubu hem core hem access switch'leri içerir
- `ansible_network_os=ios` : Cihaz işletim sistemi; doğru modülün seçilmesini sağlar
- `ansible_connection=network_cli` : SSH tabanlı CLI bağlantısı kullanılır
- `ansible_port=830` : Juniper NETCONF varsayılan portu; SSH'nin 22 yerine 830 kullanılır

Inventory Doğrulama

Inventory dosyasının doğru yapılandırıldığını kontrol edin:

```
# Tüm hostları listele
ansible-inventory -i inventory/production/hosts.ini --list

# Grafik (ağaç) görünümünde listele
ansible-inventory -i inventory/production/hosts.ini --graph

# Belirli bir grubu listele
ansible-inventory -i inventory/production/hosts.ini --host sw-core-01

# Bağlantı testi (ping)
ansible cisco -i inventory/production/hosts.ini -m ping
```

Beklenen Çıktı Örneği (ansible --graph):

```
@all:
  |--@cisco:
  |  |--@cisco_access_switches:
  |  |  |--sw-access-01
  |  |  |--sw-access-02
  |  |--@cisco_core_switches:
  |     |--sw-core-01
  |     |--sw-core-02
  |--@juniper:
  |  |--@juniper_core_switches:
  |     |--jr-core-01
  |--@switches:
  |  |--@cisco...
  |  |--@juniper...
  |--@ungrouped:
```

3.3 YAML Formatında Inventory

YAML formatı, karmaşık yapılar ve iç içe değişkenler için daha okunabilirdir. Özellikle group_vars/host_vars kullanımıyla birleştirildiğinde daha düzenli bir yapı sunar.

`inventory/production/hosts.yml`

```
all:
  children:
    cisco:
      children:
        cisco_core_switches:
          hosts:
            sw-core-01:
              ansible_host: 192.168.10.10
              site: datacenter_a
            sw-core-02:
              ansible_host: 192.168.10.11
              site: datacenter_b

        cisco_access_switches:
          hosts:
            sw-access-01:
              ansible_host: 192.168.10.101
              floor: "1"
            sw-access-02:
              ansible_host: 192.168.10.102
              floor: "2"
            sw-access-03:
              ansible_host: 192.168.10.103
              floor: "3"
      vars:
        ansible_network_os: ios
        ansible_connection: network_cli
        ansible_user: ansible
        ansible_ssh_pass: "{{ vault_cisco_pass }}"
        ansible_become: no
        ansible_become_method: enable

    juniper:
      children:
        juniper_core_switches:
          hosts:
            jr-core-01:
              ansible_host: 192.168.10.20
              site: datacenter_a
      vars:
        ansible_network_os: junos
        ansible_connection: netconf
        ansible_user: ansible
        ansible_ssh_pass: "{{ vault_juniper_pass }}"
        ansible_port: 830
```

Öneri: YAML Formatını Tercih Edin

INI formatı basit yapılar için yeterlidir ancak karmaşık ortamlarda YAML formatı şu avantajları sunar:

- Daha okunabilir hiyerarşi
- İç içe grup tanımları daha net
- Değişken şablonları (Jinja2) ile birlikte daha iyi çalışır
- IDE'ler ve editörlerde daha iyi syntax highlighting

3.4 Değişkenler ve group_vars

Büyük ortamlarda, değişkenleri doğrudan inventory dosyasına yazmak yerine ayrı dosyalarda tutmak çok daha düzenlidir. Ansible, `group_vars` ve `host_vars` dizinlerini otomatik olarak okur.

Dizin Yapısı

```
inventory/production/
├── hosts.yml
├── group_vars/
│   ├── all.yml           # Tüm cihazlar için ortak değişkenler
│   ├── cisco.yml         # Tüm Cisco cihazları için
│   ├── cisco_core_switches.yml
│   └── juniper.yml
└── host_vars/
    ├── sw-core-01.yml    # Sadece bu cihaza özel
    └── sw-access-01.yml
```

Grup Değişkenleri Örneği

`inventory/production/group_vars/cisco.yml`

```
# --- Cisco Cihazları İçin Ortak Değişkenler ---
ansible_network_os: ios
ansible_connection: network_cli
ansible_user: networkadmin
ansible_ssh_pass: "{{ vault_cisco_ssh_pass }}"
ansible_become: yes
ansible_become_method: enable
ansible_become_password: "{{ vault_cisco_enable_pass }}"

# NTP Sunucuları
ntp_servers:
  - 192.168.1.1
  - 192.168.1.2

# Syslog Sunucusu
syslog_server: 192.168.1.10

# SNMP
snmp_community: "{{ vault_snmp_community }}"
snmp_location: "Istanbul DC"

# VLAN'lar
core_vlans:
  - { id: 10, name: YONETIM }
  - { id: 20, name: SUNUCULAR }
  - { id: 30, name: KULLANICILAR }
  - { id: 40, name: MISAFIR }
  - { id: 99, name: NATIVE }
```

Cihaz Özel Değişkenler

`inventory/production/host_vars/sw-core-01.yml`

```
# --- sw-core-01 Özel Değişkenler ---
hostname: SW-CORE-01

# Loopback arayüzü
loopback0_ip: 10.0.0.1/32

# Uplink portları
uplink_ports:
  - interface: GigabitEthernet0/0
    description: "Uplink to ISP-RTR-01"
    ip_address: 172.16.1.2/30
  - interface: GigabitEthernet0/1
    description: "Uplink to ISP-RTR-02"
    ip_address: 172.16.2.2/30

# OSPF ayarları
ospf_process_id: 1
ospf_router_id: 10.0.0.1
ospf_areas:
  - area: 0
    networks:
      - 192.168.10.0/24
      - 192.168.20.0/24
```

Değişken İsimlendirme Kuralları

- Değişken isimleri harf, rakam ve alt çizgi içerebilir
- Rakamla başlayamaz
- Küçük harf ve alt çizgi kullanımı (snake_case) önerilir
- `ansible_` öneki ile başlayan isimler Ansible tarafından ayrılmıştır
- Gizli bilgiler (şifreler) için `vault_` öneki kullanılması iyi bir pratiktir

Bölüm 4: Ansible Ağ Modülleri

4.1 Bağlantı Yöntemleri

Ansible'ın ağ cihazlarıyla iletişim kurarken kullandığı bağlantı yöntemleri, hem cihazın yeteneklerine hem de otomasyon senaryonuzun karmaşıklığına bağlıdır.

Bağlantı Yöntemi	Protokol	Kullanım Alanı	Avantaj
`network_cli`	SSH (port 22)	Cisco IOS/IOS-XE/NX-OS, JunOS, EOS	En geniş cihaz desteği, basit kullanım
`netconf`	SSH üzerinden NETCONF (port 830)	Cisco IOS-XR, JunOS	Transactional işlemler, otomatik rollback
`httpapi`	HTTP/HTTPS	Cisco NX-API, RESTCONF destekleyen cihazlar	Modern API tabanlı, JSON desteği
`local`	SSH (paramiko)	Eski yöntem; önerilmez	Geriye dönük uyumluluk

4.2 ios_command ve ios_config Modülleri

Cisco IOS cihazlarında en sık kullanılan iki modül `ios_command` ve `ios_config`'tir.

ios_command Modülü

`ios_command` modülü, cihazda `show` komutları gibi salt okunur komutları çalıştırmak için kullanılır. Yapılandırma değiştirmez, sadece bilgi toplar.

```
- name: Cisco cihazdan bilgi topla
  ios_command:
    commands:
      - show version
      - show ip interface brief
      - show vlan brief
      - show clock
  register: show_output

- name: Çıktıyı görüntüle
  debug:
    var: show_output.stdout_lines
```

Kod Açıklaması

- `commands:` Çalıştırılacak komutların listesi; sırayla çalıştırılır
- `register: show_output` Komut çıktısı `show_output` değişkenine kaydedilir
- `debug:` Modülü ile değişken içeriği ekrana yazdırılır
- `stdout_lines` Çıktı satır satır listelenmiş halde erişilir

ios_config Modülü

`ios_config` modülü, cihazın yapılandırmasını değiştirmek için kullanılır. Temel parametreler:

Parametre	Açıklama
`lines`	Yapılandırma satırlarının listesi
`src`	Şablon dosyasının yolu (Jinja2)
`parents`	Alt moda girmeden önceki üst komutlar
`before`	Yapılandırmadan önce çalıştırılacak komutlar
`after`	Yapılandırmadan sonra çalıştırılacak komutlar
`save_when`	Otomatik kaydetme koşulu (modified/always/never)
`backup`	Yapılandırma öncesi yedek alma

```
- name: NTP sunucularını yapılandır
  ios_config:
    lines:
      - ntp server 192.168.1.1
      - ntp server 192.168.1.2
    save_when: modified

- name: Arayüz açıklaması ekle
  ios_config:
    lines:
      - "description Ansible yonetim arayuzu"
      - switchport mode access
      - switchport access vlan 10
      - no shutdown
    parents: interface GigabitEthernet0/1
    save_when: modified
```

Kod Açıklaması

- `lines:` Uygulanacak yapılandırma satırları; sırayla girilir
- `parents: interface GigabitEthernet0/1` Komutlar bu arayüz alt modunda çalıştırılır
- `save_when: modified` Sadece değişiklik olduğunda `write memory` çalıştırır

4.3 junos_command ve junos_config Modülleri

Juniper cihazlarında kullanılan temel modüller, Cisco modüllerine benzer yapıdadır ancak JunOS'un kendine özgü komut yapısına uygundur.

```
- name: Juniper cihazdan bilgi topla
  junos_command:
    commands:
      - show version
      - show interfaces terse
      - show vlans
    display: text
  register: junos_output

- name: Versiyon bilgisini göster
  debug:
    msg: "Cihaz versiyonu: {{ junos_output.stdout[0] }}"
```

```
- name: VLAN yapılandırması ekle
  junos_config:
    lines:
      - "set vlans YONETIM vlan-id 10"
      - "set vlans YONETIM description Yonetim VLAN'i"
      - "set vlans SATIS vlan-id 100"
    comment: "Ansible: VLAN'lar eklendi"

- name: Yapılandırmayı kaydet (5 dk rollback ile)
  junos_config:
    commit: yes
    confirm: 5
```

Kod Açıklaması

- `display: text` Çıktıyı XML yerine düz metin olarak alır
- `set vlans ...` JunOS yapılandırma modunda set komutları kullanılır
- `comment:` JunOS commit mesajı olarak kaydedilir; denetim izi bırakır
- `confirm: 5` 5 dakika içinde onaylanmazsa otomatik rollback yapar

Önemli: JunOS Confirm Mekanizması

`confirm: 5` parametresi, yapılandırmanızı otomatik olarak geri alacak bir güvenlik süresi ekler. Yapılandırmayı uyguladıktan sonra cihaza hala erişebiliyorsanız, başka bir playbook veya manuel olarak `commit` komutunu çalıştırarak kalıcı hale getirebilirsiniz. Bu özellik, özellikle uzak veri merkezlerinde çalışırken hayat kurtarıcıdır.

4.4 Modül Seçim Rehberi

İşlem	Cisco IOS	Juniper JunOS
Bilgi toplama	`ios_facts`	`junos_facts`
Komut çalıştırma	`ios_command`	`junos_command`
Yapılandırma	`ios_config`	`junos_config`
VLAN yönetimi	`ios_vlan`	`junos_config` (set)

Arayüz yönetimi	`ios_interface`	`junos_config` (set)
Kullanıcı yönetimi	`ios_user`	`junos_user`
Yapılandırma kaydetme	`ios_config (save_when)`	Otomatik (commit)

İpucu: Modül Dokümantasyonuna Erişim

Herhangi bir modülün tüm parametrelerini ve örneklerini görmek için:

```
ansible-doc ios_config
ansible-doc junos_config
```

Bölüm 5: Playbook #1 - Toplu VLAN Oluşturma

5.1 Senaryo Tanımı

Bu senaryoda, birden fazla Cisco ve Juniper switch'e toplu olarak VLAN tanımlayacağız. Ağ topolojimizde core ve access katmanında toplam 5 switch bulunmaktadır. Yeni bir departman için 4 yeni VLAN oluşturulması gerekmektedir.

> **Senaryo Detayları**
>
> - **Hedef:** Tüm switch'lere tutarlı VLAN yapılandırması uygulamak
> - **Kapsam:** Core ve access katmanındaki tüm Cisco ve Juniper switch'ler
> - **VLAN'lar:** YONETIM(10), SATIS(100), MUHASEBE(200), IT(300)
> - **Kısıtlar:** Mevcut VLAN'lar silinmeyecek, sadece eksik olanlar eklenecek
> - **Raporlama:** İşlem sonucu hangi VLAN'ların eklendiği raporlanacak

Gereksinimler ve Değişkenler

Öncelikle playbook'da kullanılacak değişkenleri tanımlayalım:

`group_vars/all.yml`

```
# --- Toplu VLAN Değişkenleri ---
new_vlans:
  - vlan_id: 10
    vlan_name: "YONETIM"
  - vlan_id: 100
    vlan_name: "SATIS"
  - vlan_id: 200
    vlan_name: "MUHASEBE"
  - vlan_id: 300
    vlan_name: "IT"

# VLAN aralığı kontrolü için
allowed_vlan_range: "10,100,200,300"
```

5.2 Cisco IOS VLAN Yönetimi

Playbook: playbooks/vlan-management.yml

Cisco ve Juniper switch'lere toplu VLAN ekleme (idempotent)

```
---
- name: "PLAY 1: Cisco Switch'lere VLAN Yapilandirma"
  hosts: cisco_switches
  gather_facts: no
  connection: network_cli

  vars:
    vlan_list:
      - { id: 10, name: "YONETIM" }
      - { id: 100, name: "SATIS" }
      - { id: 200, name: "MUHASEBE" }
      - { id: 300, name: "IT" }

  tasks:
    # ========================================
    # GÖREV 1: Mevcut VLAN'lari kontrol et
    # ========================================
    - name: "Mevcut VLAN listesini al"
      ios_command:
        commands: "show vlan brief"
      register: vlan_brief_output

    # ========================================
    # GÖREV 2: VLAN'lari yapilandir
    # ========================================
    - name: "VLAN'lari oluştur veya guncelle"
      ios_vlan:
        vlan_id: "{{ item.id }}"
        name: "{{ item.name }}"
        state: present
      loop: "{{ vlan_list }}"
      register: vlan_result

    # ========================================
    # GÖREV 3: Degisiklik ozeti
    # ========================================
    - name: "VLAN degisiklik ozeti"
      debug:
                  msg:  "VLAN  {{  item.id  }}  ({{  item.name  }}):  {{
vlan_result.results[loop0].changed   |   ternary('OLUSTURULDU/GUNCELLENDI',
'ZATEN MEVCUT') }}"
      loop: "{{ vlan_list }}"
      loop_control:
        index_var: loop0

    # ========================================
    # GÖREV 4: Yapilandirmayi kaydet
```

Kod Açıklaması (Satır Satır)

- `hosts: cisco_switches` Bu play, inventory'deki "cisco_switches" grubundaki tüm cihazlarda çalışır
- `gather_facts: no` Sunucu facts'lerini toplamaz; ağ cihazlarında varsayılan facts toplama anlamsızdır ve zaman alır
- `connection: network_cli` SSH tabanlı CLI bağlantısı kullanılır
- `vars:` Playbook seviyesinde değişken tanımlanır; inventory'deki değişkenlerin üzerine yazar
- `ios_command: show vlan brief` Mevcut VLAN'ları görüntüler; ileride karşılaştırma için kullanılabilir
- `ios_vlan:` Modülü, VLAN yönetimini idempotent olarak yapar; VLAN zaten varsa dokunmaz
- `loop: "{{ vlan_list }}"` Her VLAN için görevi tekrarlar; 4 VLAN = 4 kez çalışır
- `register: vlan_result` Her görevin sonucunu kaydeder; changed/ok/failed durumunu içerir
- `ternary('OLUSTURULDU', 'ZATEN MEVCUT')` Jinja2 filtresi; koşula göre iki değerden birini seçer
- `when: vlan_result.changed` Sadece değişiklik olduğunda yapılandırmayı kaydeder
- `tags: save_config` Sadece belirli etiketlerle çalıştırılabilir: `--tags save_config`

Çalıştırma ve Çıktı

```
# Playbook'u çalıştır
ansible-playbook -i inventory/production/hosts.ini playbooks/vlan-
management.yml

# Sadece Cisco switch'leri hedefle
ansible-playbook -i inventory/production/hosts.ini playbooks/vlan-
management.yml --limit cisco_switches

# Sadece kaydetme adımını atla
ansible-playbook -i inventory/production/hosts.ini playbooks/vlan-
management.yml --skip-tags save_config

# Fazla ayrıntı ile çalıştır
ansible-playbook -i inventory/production/hosts.ini playbooks/vlan-
management.yml -vvv
```

5.3 Juniper JunOS VLAN Yönetimi

Juniper cihazlarında VLAN yapılandırması, Cisco'dan farklı olarak `vlans` hiyerarşisi altında tanımlanır. İşte JunOS için karşılık playbook:

```
---
- name: "PLAY 2: Juniper Switch'lere VLAN Yapilandirma"
  hosts: juniper_switches
  gather_facts: no
  connection: netconf

  vars:
    vlan_list:
      - { id: 10, name: "YONETIM", description: "Yonetim birimi" }
      - { id: 100, name: "SATIS", description: "Satis departmani" }
      - { id: 200, name: "MUHASEBE", description: "Muhasebe departmani" }
      - { id: 300, name: "IT", description: "IT departmani" }

  tasks:
    # ========================================
    # GÖREV 1: Mevcut VLAN yapılandırmasını al
    # ========================================
    - name: "Mevcut VLAN'lari kontrol et"
      junos_command:
        commands: "show vlans"
        display: text
      register: existing_vlans

    # ========================================
    # GÖREV 2: VLAN'lari set komutlari ile olustur
    # ========================================
    - name: "VLAN'lari JunOS'a ekle"
      junos_config:
        lines:
          - "set vlans {{ item.name }} vlan-id {{ item.id }}"
            - "set vlans {{ item.name }} description \"{{ item.description
}}\""
        comment: "Ansible: VLAN {{ item.name }} ({{ item.id }}) eklendi"
      loop: "{{ vlan_list }}"
      register: vlan_config_result

    # ========================================
    # GÖREV 3: Degisiklik ozeti
    # ========================================
    - name: "Islem ozeti"
      debug:
        msg: |
          Cihaz: {{ inventory_hostname }}
            Isleme alinan VLAN'lar: {{ vlan_list | map(attribute='name') |
join(', ') }}
          Degisiklik yapildi: {{ vlan_config_result.changed }}
```

Kod Açıklaması (Satır Satır)

- `connection: netconf` JunOS için NETCONF protokolü kullanılır; port 830
- `junos_config: lines:` Her VLAN için 2 `set` komutu çalıştırılır
- `comment:` JunOS commit mesajı olarak kaydedilir; denetim izi bırakır
- `commit: yes` Yapılandırmayı aktif hale getirir; JunOS'ta `commit` gereklidir
- `confirm: 5` 5 dakika içinde onaylanmazsa otomatik rollback yapar
- `map(attribute='name') | join(', ')` VLAN isimlerini virgülle birleştirir

İpucu: idempotency Kontrolü

`ios_vlan` ve `junos_config` modülleri idempotent'tir. Aynı playbook'u tekrar çalıştırırsanız, zaten var olan VLAN'lar için "ok" (değişiklik yok) mesajı alırsınız. Sadece eksik VLAN'lar eklenecektir.

5.4 Adım Adım Açıklama

Bu playbook'un çalışma mantığını adım adım inceleyelim:

1 **Inventory Okuma**

Ansible inventory dosyasını okur ve `cisco_switches` grubundaki cihazları belirler. Her cihaz için `ansible_host`, `ansible_user`, `ansible_network_os` gibi değişkenleri yükler.

2 **Bağlantı Kurma**

Her cihaza SSH (Cisco) veya NETCONF (Juniper) bağlantısı kurulur. `ansible_connection` değişkenine göre doğru protokol seçilir.

3 **Mevcut Durumu Kontrol Etme**

`show vlan brief` komutu çalıştırılarak mevcut VLAN listesi alınır. Bu bilgi ileride karşılaştırma için kullanılabilir.

4 **VLAN Oluşturma (Loop)**

`loop` mekanizması ile her VLAN için `ios_vlan` modülü çalıştırılır. Modül, VLAN'ın zaten olup olmadığını kontrol eder (idempotent).

5 **Sonuç Raporlama**

`debug` modülü ile hangi VLAN'ların oluşturulduğu/güncellendiği, hangilerinin zaten mevcut olduğu raporlanır.

6 **Yapılandırmayı Kaydetme**

Eğer değişiklik olduysa (`when: vlan_result.changed`), yapılandırma kalıcı hale getirilir. Cisco'da `write memory` , Juniper'da `commit` ile.

Bölüm 6: Playbook #2 - Konfigürasyon Yedekleme

6.1 Senaryo Tanımı

Bu senaryoda, tüm ağ cihazlarının yapılandırmalarını otomatik olarak yedekleyeceğiz. Yedekleme, ağ yönetiminin en kritik görevlerinden biridir. Manuel yedekleme unutulabilir veya tutarsız olabilir; otomasyon bu riski ortadan kaldırır.

Senaryo Detayları

- **Hedef:** Tüm switch ve router'lardan yapılandırma yedeği almak
- **Çıktı:** Her cihaz için tarih-saat damgalı dosya
- **Saklama:** Son 30 günün yedekleri tutulacak
- **Raporlama:** Yedekleme başarı durumu raporlanacak
- **Format:** Düz metin (.cfg) formatında

6.2 Cisco Konfigürasyon Yedekleme

Playbook: playbooks/config-backup.yml

Tüm cihazlardan yapılandırma yedeği alma ve rotasyon yönetimi

```
---
- name: "PLAY 1: Cisco Cihazlari Yedekleme"
  hosts: cisco_devices
  gather_facts: no
  connection: network_cli

  vars:
    backup_dir: "./outputs/backups"
    backup_retention_days: 30
    timestamp: "{{ ansible_date_time.date }}_{{ ansible_date_time.hour }}{{
ansible_date_time.minute }}"

  tasks:
    # ========================================
    # GÖREV 1: Yedekleme dizinini olustur
    # ========================================
    - name: "Yedekleme dizinini kontrol et"
      file:
        path: "{{ backup_dir }}/cisco"
        state: directory
        mode: '0755'
      delegate_to: localhost
      run_once: yes

    # ========================================
    # GÖREV 2: Running-config'i cihazdan al
    # ========================================
    - name: "Running-config'i al"
      ios_command:
        commands:
          - "show running-config"
      register: running_config

    # ========================================
    # GÖREV 3: Yapılandırmayi dosyaya kaydet
    # ========================================
    - name: "Yapılandırmayi dosyaya yaz"
      copy:
        content: "{{ running_config.stdout[0] }}"
        dest: "{{ backup_dir }}/cisco/{{ inventory_hostname }}_{{ timestamp
}}.cfg"
        mode: '0644'
      delegate_to: localhost
      register: backup_file

    # ========================================
    # GÖREV 4: Son yedek symlink'i guncelle
```

Kod Açıklaması (Satır Satır)

- `delegate_to: localhost` Dosya işlemleri Ansible kontrol düğümünde yapılır; cihaza dosya yazılmaz
- `run_once: yes` Dizin oluşturma ve rapor görevleri sadece bir kez çalışır
- `ansible_date_time` Ansible'ın otomatik tarih/değişkeni; çalışma anının zamanı
- `running_config.stdout[0]` `show running-config` çıktısının tamamı
- `state: link` + `force: yes` Son yedeğe sabit bir symlink oluşturur; script'ler `_latest.cfg` dosyasını kullanabilir
- `find ... -mtime +30 -delete` 30 günden eski yedek dosyalarını siler
- `post_tasks:` Play'in tüm task'ları bittikten sonra çalışır; toplu rapor için idealdir

Jinja2 Rapor Şablonu

`templates/backup_report.j2`

```
{# backup_report.j2 - Yedekleme Raporu Şablonu #}
============================================================
  AĞ CİHAZI YEDEKLEME RAPORU
============================================================
Tarih: {{ ansible_date_time.iso8601 }}
Toplam Cihaz: {{ ansible_play_hosts | length }}
Basarili:  {{  ansible_play_hosts  |  difference(ansible_play_hosts_all)  |
length }}
Basarisiz: {{ ansible_failed_hosts | default([]) | length }}

------------------------------------------------------------
CIHAZ LISTESI
------------------------------------------------------------
{% for host in ansible_play_hosts %}
- {{ host }}: {{ hostvars[host]['backup_file']['dest'] | default('Hata') }}
{% endfor %}

============================================================
```

6.3 Juniper Konfigürasyon Yedekleme

Juniper cihazlarında yapılandırma yedekleme, `junos_config` modülünün yerleşik `backup` parametresi ile çok daha kolaydır:

```
---
- name: "PLAY 2: Juniper Cihazlari Yedekleme"
  hosts: juniper_devices
  gather_facts: no
  connection: netconf

  vars:
    backup_dir: "./outputs/backups"
    timestamp: "{{ ansible_date_time.date }}_{{ ansible_date_time.hour }}{{
ansible_date_time.minute }}"

  tasks:
    # ========================================
    # GÖREV 1: Yedekleme dizinini olustur
    # ========================================
    - name: "Yedekleme dizinini kontrol et"
      file:
        path: "{{ backup_dir }}/juniper"
        state: directory
        mode: '0755'
      delegate_to: localhost
      run_once: yes

    # ========================================
    # GÖREV 2: JunOS yapılandırmasını yedekle
    # ========================================
    - name: "Aktif yapılandırmayi yedekle"
      junos_config:
        backup: yes
        backup_options:
          filename: "{{ inventory_hostname }}_{{ timestamp }}.cfg"
          dir_path: "{{ backup_dir }}/juniper"
      register: backup_result

    # ========================================
    # GÖREV 3: set formatinda da al (tercih)
    # ========================================
    - name: "Yapılandırmayi set formatinda al"
      junos_command:
        commands: "show configuration | display set"
        display: text
      register: config_set_format

    - name: "Set formatini dosyaya kaydet"
      copy:
        content: "{{ config_set_format.stdout[0] }}"
             dest:  "{{  backup_dir  }}/juniper/{{  inventory_hostname  }}_{{
```

Kod Açıklaması

- `backup: yes` JunOS `junos_config` modülü yerleşik yedekleme desteği sunar; XML formatında yedek alır
- `backup_options:` Yedek dosya adı ve dizinini özelleştirir
- `show configuration | display set` JunOS yapılandırmasını "set" komutları formatında verir; restore etmek daha kolaydır
- İki format alınır: XML (tam yapılandırma) ve set (okunabilir/restore edilebilir)

6.4 Otomatik Zamanlamalı Yedekleme

Yedekleme playbook'unu düzenli olarak çalıştırmak için `cron` kullanabilirsiniz:

Crontab ayarı

```
# Her gece saat 02:00'da yedekleme yap
0 2 * * * cd ~/network-automation && /usr/bin/ansible-playbook -i inventory/production/hosts.ini playbooks/config-backup.yml >> outputs/backup_cron.log 2>&1

# Her Pazartesi sabah rapor e-posta ile gönder
0 8 * * 1 cd ~/network-automation && cat outputs/backups/report_$(date +\%Y-\%m-\%d).txt | mail -s "Haftalik Yedekleme Raporu" ag-yoneticisi@sirket.com
```

Öneri: Ansible Tower / AWX ile Zamanlama

Kurumsal ortamlarda crontab yerine **Ansible Tower** veya açık kaynak alternatifi **AWX** kullanarak:

- Web arayüzünden zamanlama yönetimi
- Başarısız işlem bildirimleri (e-posta/Slack)
- İşlem geçmişi ve audit kayıtları
- Rol bazlı erişim kontrolü (RBAC)

Bölüm 7: Playbook #3 - Arayüz Yapılandırması

7.1 Senaryo Tanımı

Bu senaryoda, switch portlarının toplu olarak yapılandırılması işlemini otomatikleştireceğiz. Yeni bir katın devreye alınması senaryosu üzerinden gidelim: 24 portlu 3 adet access switch'e yeni kullanıcılar için port yapılandırması uygulanacaktır.

Senaryo Detayları

- **Hedef:** 72 portun (3 switch x 24 port) yapılandırılması
- **Port modu:** Access (trunk portlar hariç)
- **VLAN atama:** Bölüm bazlı (SATIS:100, MUHASEBE:200, IT:300)
- **Port güvenliği:** Port-security etkinleştirme
- **Port açıklaması:** Kullanıcı adı ve bölüm bilgisi
- **Spanning Tree:** PortFast etkinleştirme

Veri Yapısı

Her portun yapılandırmasını bir veri yapısında tanımlayalım:

`host_vars/sw-access-01.yml`

```
# --- sw-access-01 Arayüz Yapılandırması ---
interfaces:
  - name: GigabitEthernet1/0/1
    description: "AHMET.YILMAZ - SATIS"
    mode: access
    access_vlan: 100
    port_security: yes
    max_mac: 2
    spanning_tree_portfast: yes

  - name: GigabitEthernet1/0/2
    description: "AYSE.DEMIR - MUHASEBE"
    mode: access
    access_vlan: 200
    port_security: yes
    max_mac: 2
    spanning_tree_portfast: yes

  - name: GigabitEthernet1/0/3
    description: "MEHMET.KAYA - IT"
    mode: access
    access_vlan: 300
    port_security: yes
    max_mac: 3
    spanning_tree_portfast: yes

  # ... (diger portlar)

  - name: GigabitEthernet1/0/24
    description: "UPLINK-to-CORE"
    mode: trunk
    trunk_allowed_vlans: "10,100,200,300"
    trunk_native_vlan: 99
    spanning_tree_portfast: no
```

7.2 Cisco Arayüz Yapılandırması

Playbook: playbooks/interface-config.yml

Access switch portlarinin toplu yapilandirilmasi

```
---
- name: "PLAY 1: Cisco Access Switch Port Yapilandirmasi"
  hosts: cisco_access_switches
  gather_facts: no
  connection: network_cli

  tasks:
    # ========================================
    # GÖREV 1: Access portlari yapilandir
    # ========================================
    - name: "Access portlarini yapilandir"
      ios_config:
        lines:
          - "description {{ item.description }}"
          - switchport mode access
          - "switchport access vlan {{ item.access_vlan }}"
          - spanning-tree portfast
          - no shutdown
        parents: "interface {{ item.name }}"
        save_when: modified
      loop: "{{ interfaces }}"
      when: item.mode == 'access'
      register: access_config

    # ========================================
    # GÖREV 2: Trunk portlari yapilandir
    # ========================================
    - name: "Trunk portlarini yapilandir"
      ios_config:
        lines:
          - "description {{ item.description }}"
          - switchport mode trunk
          - "switchport trunk allowed vlan {{ item.trunk_allowed_vlans }}"
          - "switchport trunk native vlan {{ item.trunk_native_vlan }}"
          - no shutdown
        parents: "interface {{ item.name }}"
        save_when: modified
      loop: "{{ interfaces }}"
      when: item.mode == 'trunk'
      register: trunk_config

    # ========================================
    # GÖREV 3: Port-security yapilandirmasi
    # ========================================
    - name: "Port-security etkinlestir"
      ios_config:
        lines:
```

Kod Açıklaması (Satır Satır)

- `loop: "{{ interfaces }}"` Her port için görev tekrarlanır; interfaces değişkeni host_vars'dan gelir
- `when: item.mode == 'access'` Sadece access modundaki portlarda çalışır; trunk portlar atlanır
- `parents: "interface {{ item.name }}"` Her komut öncesinde ilgili arayüz alt moduna girilir
- `port_security | default(false) | bool` Port-security tanımlanmamışsa varsayılan false; boolean tipine çevrilir
- `violation restrict` İhlal durumunda portu kapatmak yerine paketleri düşürür ve log üretir
- `mac-address sticky` Öğrenilen ilk MAC adresini kalıcı olarak kaydeder
- `selectattr('changed', 'equalto', true)` Jinja2 filtresi; sadece değişen sonuçları sayar

Çalıştırma ve Kontrol

```
# Playbook'u çalıştır
ansible-playbook -i inventory/production/hosts.ini playbooks/interface-config.yml

# Sadece bir switch'i hedefle
ansible-playbook -i inventory/production/hosts.ini playbooks/interface-config.yml --limit sw-access-01

# Değişiklik yapmadan kontrol et (--check = dry-run)
ansible-playbook -i inventory/production/hosts.ini playbooks/interface-config.yml --check

# Fazla detay ile çalıştır
ansible-playbook -i inventory/production/hosts.ini playbooks/interface-config.yml -v
```

Önemli: --check Modu (Dry-Run)

`--check` parametresi, playbook'un değişiklik yapmadan ne yapacağını gösterir. Ancak bazı modüller tam dry-run desteği sunmayabilir. Her zaman test ortamında deneyin.

7.3 Juniper Arayüz Yapılandırması

Juniper cihazlarında arayüz yapılandırması, unit ve family hiyerarşisi ile farklılık gösterir:

```
---
- name: "PLAY 2: Juniper Access Switch Port Yapilandirmasi"
  hosts: juniper_access_switches
  gather_facts: no
  connection: netconf

  vars:
    interfaces:
      - name: ge-0/0/1
        description: "AHMET.YILMAZ - SATIS"
        mode: access
        vlan: SATIS

      - name: ge-0/0/2
        description: "AYSE.DEMIR - MUHASEBE"
        mode: access
        vlan: MUHASEBE

      - name: ge-0/0/24
        description: "UPLINK-to-CORE"
        mode: trunk
        trunk_vlans: [ "YONETIM", "SATIS", "MUHASEBE", "IT" ]
        native_vlan: "NATIVE"

  tasks:
    # ========================================
    # GÖREV 1: Access portlari yapilandir
    # ========================================
    - name: "Access portlarini yapilandir"
      junos_config:
        lines:
                    - "set interfaces {{ item.name }} description \"{{
item.description }}\""
                - "set interfaces {{ item.name }} unit 0 family ethernet-
switching interface-mode access"
                - "set interfaces {{ item.name }} unit 0 family ethernet-
switching vlan members {{ item.vlan }}"
        comment: "Ansible: {{ item.name }} access yapilandirmasi"
      loop: "{{ interfaces }}"
      when: item.mode == 'access'
      register: junos_access

    # ========================================
    # GÖREV 2: Trunk portlari yapilandir
    # ========================================
    - name: "Trunk portlarini yapilandir"
      junos_config:
```

Kod Açıklaması

- `unit 0` JunOS'ta her fiziksel arayüz en az bir logical unit'a sahiptir; ethernet-switching için unit 0 kullanılır
- `family ethernet-switching` L2 switching modunu belirtir
- `interface-mode access` Port modunu access olarak ayarlar
- `vlan members {{ item.vlan }}` Portun üyesi olduğu VLAN'ı atar
- `item.trunk_vlans | join(' ')` VLAN listesini boşlukla birleştirir; JunOS set komutu bu formatta ister
- `storm-control default` Broadcast/unknown unicast/multicast storm koruması ekler

7.4 Jinja2 Şablonları ile Dinamik Yapılandırma

Büyük ortamlarda her portu tek tek tanımlamak yerine, Jinja2 şablonları ile dinamik yapılandırma oluşturabilirsiniz. Bu yaklaşım, özellikle benzer yapılandırmaların toplu uygulanmasında çok etkilidir.

Dinamik Port Üretimi (CSV'den)

Bir CSV dosyasından port bilgilerini okuyarak yapılandırma üretelim:

`files/port_list.csv`

```
switch,port,description,vlan,mode
sw-access-01,GigabitEthernet1/0/1,AHMET.YILMAZ - SATIS,100,access
sw-access-01,GigabitEthernet1/0/2,AYSE.DEMIR - MUHASEBE,200,access
sw-access-01,GigabitEthernet1/0/3,MEHMET.KAYA - IT,300,access
sw-access-02,GigabitEthernet1/0/1,FATMA.SAHIN - SATIS,100,access
sw-access-02,GigabitEthernet1/0/2,MUSTAFA.CELIK - MUHASEBE,200,access
```

`playbooks/interface-config-from-csv.yml`

```
---
- name: "CSV'den Port Yapilandirmasi"
  hosts: cisco_access_switches
  gather_facts: no
  connection: network_cli

  tasks:
    # ========================================
    # GÖREV 1: CSV dosyasini oku
    # ========================================
    - name: "CSV dosyasini oku"
      read_csv:
        path: "files/port_list.csv"
      delegate_to: localhost
      register: port_list

    # ========================================
    # GÖREV 2: Bu switch'e ait portlari filtrele
    # ========================================
    - name: "Portlari filtrele"
      set_fact:
           my_ports: "{{ port_list.list | selectattr('switch', 'equalto',
inventory_hostname) | list }}"

    # ========================================
    # GÖREV 3: Portlari yapilandir
    # ========================================
    - name: "Portlari yapilandir"
      ios_config:
        lines:
          - "description {{ item.description }}"
          - switchport mode access
          - "switchport access vlan {{ item.vlan }}"
          - spanning-tree portfast
          - no shutdown
        parents: "interface {{ item.port }}"
        save_when: modified
      loop: "{{ my_ports }}"
      when: my_ports | length > 0

    # ========================================
    # GÖREV 4: Ozet rapor
    # ========================================
    - name: "Yapilandirma ozeti"
      debug:
           msg: "{{ inventory_hostname }}: {{ my_ports | length }} port
```

Kod Açıklaması

- `read_csv:` Modülü CSV dosyasını okur; her satır bir dictionary olur
- `delegate_to: localhost` CSV dosyası kontrol düğümünde okunur
- `selectattr('switch', 'equalto', inventory_hostname)` Sadece mevcut cihaza ait portları filtreler
- `set_fact:` Filtrelenmiş listeyi `my_ports` değişkenine kaydeder; tekrar tekrar filtrelemez
- CSV yaklaşımı, yüzlerce portun yönetimini Excel'den kolayca yapmayı sağlar

Jinja2 Şablon ile Tam Yapılandırma

Karmaşık yapılandırmalar için tam bir şablon kullanabilirsiniz:

`templates/cisco_interface_template.j2`

```
{# Cisco Interface Yapilandirma Şablonu #}
{% for intf in interfaces %}
interface {{ intf.name }}
 description {{ intf.description }}
{% if intf.mode == 'access' %}
 switchport mode access
 switchport access vlan {{ intf.access_vlan }}
 spanning-tree portfast
{% if intf.port_security | default(false) %}
 switchport port-security
 switchport port-security maximum {{ intf.max_mac | default(1) }}
 switchport port-security violation restrict
 switchport port-security mac-address sticky
{% endif %}
{% elif intf.mode == 'trunk' %}
 switchport mode trunk
 switchport trunk allowed vlan {{ intf.trunk_allowed_vlans }}
 switchport trunk native vlan {{ intf.trunk_native_vlan | default(1) }}
{% endif %}
 no shutdown
!
{% endfor %}
```

Şablonu kullanan playbook:

```
- name: "Jinja2 şablonundan yapilandirma uygula"
  ios_config:
    src: templates/cisco_interface_template.j2
    backup: yes
    save_when: modified
```

İpucu: Şablon Test Etme

Jinja2 şablonunuzun çıktısını önceden görmek için:

```
ansible-playbook playbooks/interface-config.yml --check -v
```

Bölüm 8: İleri Seviye ve Best Practices

8.1 Hata Yakalama ve Retry Mekanizmaları

Ağ otomasyonunda hatalar kaçınılmazdır: bir cihaz yanıt vermeyebilir, SSH bağlantısı zaman aşımına uğrayabilir veya yapılandırma komutu hatalı olabilir. Bu durumları zarif bir şekilde yönetmek için Ansible çeşitli mekanizmalar sunar.

Block-Rescue-Always Yapısı

Try-catch benzeri bu yapı, hata durumlarında kurtarma işlemleri yapmanızı sağlar:

```
- name: "Guvenli VLAN yapilandirmasi"
  block:
    - name: "VLAN'lari olustur"
      ios_vlan:
        vlan_id: "{{ item.id }}"
        name: "{{ item.name }}"
        state: present
      loop: "{{ new_vlans }}"

    - name: "Portlari VLAN'a ata"
      ios_config:
        lines:
          - "switchport access vlan {{ item.vlan }}"
        parents: "interface {{ item.port }}"
      loop: "{{ port_assignments }}"

  rescue:
    - name: "HATA: Yapilandirmayi geri al"
      ios_config:
        lines:
          - "no vlan {{ item.id }}"
      loop: "{{ new_vlans }}"

    - name: "Hata bildirimi"
      debug:
        msg: "Yapilandirma basarisiz oldu ve geri alindi!"

  always:
    - name: "Her durumda log kaydi olustur"
      lineinfile:
        path: "./outputs/vlan_changes.log"
        line: "{{ ansible_date_time.iso8601 }} - {{ inventory_hostname }} -
VLAN islemi tamamlandi"
        create: yes
      delegate_to: localhost
```

Kod Açıklaması

- `block:` Normal iş akışı; hata olursa rescue bölümüne atlar
- `rescue:` Block içinde herhangi bir görev başarısız olursa çalışır; geri alma işlemleri buraya yazılır
- `always:` İster başarılı ister başarısız olsun, her durumda çalışır; temizlik ve loglama için idealdir

Retry Mekanizması

Yavaş cihazlar veya geçici ağ sorunları için tekrar deneme mekanizması:

```
- name: "Yapilandirmayi uygula (retry ile)"
  ios_config:
    lines:
      - "ntp server 192.168.1.1"
      - "ntp server 192.168.1.2"
    save_when: modified
  register: config_result
  until: config_result is succeeded
  retries: 3
  delay: 10

- name: "Cihazdan bilgi topla (retry ile)"
  ios_command:
    commands: "show version"
  register: version_info
  until: version_info is succeeded
  retries: 5
  delay: 5
```

Kod Açıklaması

- `register:` Görev sonucunu değişkene kaydeder; başarı/başarısızlık durumunu içerir
- `until: config_result is succeeded` Başarılı olana kadar dener
- `retries: 3` Maksimum 3 kez tekrar dener (ilk deneme + 2 retry)
- `delay: 10` Her deneme arasında 10 saniye bekler

8.2 Vault ile Şifreli Değişkenler

Playbook'larda şifre, API anahtarı gibi hassas bilgileri düz metin olarak tutmak güvenlik riski taşır. Ansible Vault bu verileri şifrelemenizi sağlar.

Vault Dosyası Oluşturma

```
# Yeni vault dosyası oluştur (interaktif şifre)
ansible-vault create group_vars/all/vault.yml

# Mevcut dosyayı vault ile şifrele
ansible-vault encrypt group_vars/all/credentials.yml

# Vault dosyasını düzenle
ansible-vault edit group_vars/all/vault.yml

# Şifreyi çöz
ansible-vault decrypt group_vars/all/vault.yml

# Şifreyi değiştir
ansible-vault rekey group_vars/all/vault.yml
```

Vault İçeriği Örneği

`group_vars/all/vault.yml (şifrelenmiş)`

```
vault_cisco_ssh_user: "networkadmin"
vault_cisco_ssh_pass: "S3cur3P@ssw0rd!"
vault_cisco_enable_pass: "En@b13S3cr3t!"
vault_juniper_ssh_pass: "Jun1p3r#2024"
vault_snmp_community: "C0mmun1tyStr1ng"
vault_ansible_become_pass: "B3c0m3S3cr3t"
```

Şifresiz Değişkenlerden Vault'a Referans

`group_vars/all/main.yml`

```
# Vault değişkenlerini normal değişkenlere ata
ansible_user: "{{ vault_cisco_ssh_user }}"
ansible_ssh_pass: "{{ vault_cisco_ssh_pass }}"
ansible_become_password: "{{ vault_cisco_enable_pass }}"
```

Vault ile Playbook Çalıştırma

```
# Interaktif şifre sor
ansible-playbook -i inventory/hosts.ini playbooks/vlan-management.yml --
ask-vault-pass

# Şifre dosyası kullan (CI/CD için ideal)
ansible-playbook -i inventory/hosts.ini playbooks/vlan-management.yml --
vault-password-file ~/.ansible/vault_pass.txt

# Şifreyi ortam değişkeninden al
export ANSIBLE_VAULT_PASSWORD_FILE=~/.ansible/vault_pass.txt
ansible-playbook -i inventory/hosts.ini playbooks/vlan-management.yml
```

Güvenlik Uyarıları

- Vault şifre dosyasını asla Git reposuna eklemeyin (`.gitignore`)
- Şifre dosyası izinlerini sıkı tutun: `chmod 600 vault_pass.txt`
- Farklı ortamlar (prod/staging) için farklı vault şifreleri kullanın
- Şifreleri düzenli olarak değiştirin (90 günde bir)

8.3 Idempotency ve Dry-Run

Ansible'ın en güçlü özelliklerinden biri **idempotency**'dir. Aynı playbook'u kaç kez çalıştırırsanız çalıştırın, sistem istenen durumdaysa değişiklik yapılmaz.

Idempotency Prensipleri

Prensip	Açıklama
`state: present`	Var olmayanı ekle, varsa dokunma
`state: absent`	Varsa kaldır, yoksa dokunma
`replace: config`	Tüm bloğu değiştir
`match: exact`	Tam eşleşme kontrolü
`match: strict`	Satır sıralı eşleşme

`match: line`	Satır bazlı eşleşme

Dry-Run (--check) Kullanımı

```
# Once dry-run ile kontrol et
ansible-playbook playbooks/vlan-management.yml --check

# Dry-run + diff (ne değişeceğini göster)
ansible-playbook playbooks/vlan-management.yml --check --diff

# Diff ile gerçek çalıştırma
ansible-playbook playbooks/vlan-management.yml --diff
```

changed_when ile Çıktı Kontrolü

```
- name: "Cihazdan show komutu calistir"
  ios_command:
    commands: "show version"
  register: version_output
  changed_when: false

- name: "Yapilandirmayi kontrol et"
  ios_config:
    lines: "ntp server 192.168.1.1"
  register: ntp_config
    changed_when:  "'already  configured'  not  in  ntp_config.stdout[0]  |
default('')"
```

Kod Açıklaması

- `changed_when: false` Görevi her zaman "değişiklik yok" (ok) olarak işaretler
- `changed_when: "'already configured' not in ..."` Çıktıda "already configured" varsa değişiklik yok sayar
- Bu, playbook çıktısının daha okunabilir olmasını sağlar

8.4 Performans Optimizasyonu

Büyük ağ ortamlarında (100+ cihaz) performans kritik hale gelir. Aşağıdaki teknikler ile playbook çalışma süresini önemli ölçüde kısaltabilirsiniz.

Paralel İşleme (Forks)

```
# Komut satırında forks ayarlama
ansible-playbook playbooks/vlan-management.yml -f 20

# ansible.cfg'de forks ayarlama
[defaults]
forks = 20
```

Dikkat: Forks Sınırı

- Çok yüksek forks değerleri, cihazın SSH daemon'ını aşırı yükleyebilir
- Başlangıç olarak `forks = 10` deneyin
- Cihazların CPU/bellek kullanımını izleyin

Serial Kullanımı (Kontrollü Yuvarlanma)

```
# Once 2 cihaz, sonra digerleri
- hosts: all
  serial: 2
  tasks:
    - ios_config:
        lines: "vlan 100"

# Yuzde olarak
- hosts: all
  serial: "10%"
  tasks:
    - ios_vlan:
        vlan_id: 100
        name: "TEST"

# Artan oranli
- hosts: all
  serial:
    - 1
    - 5
    - 10
    - 25%
```

Diğer Optimizasyonlar

```
[persistent_connection]
connect_timeout = 30
command_timeout = 30
connect_retry_timeout = 15

[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

```
- hosts: cisco_devices
  gather_facts: no  # Facts toplamayi devre disi birak
  tasks:
    - ...
```

Performans Karşılaştırması

Yöntem	50 Cihaz Süresi
Varsayılan (forks=5)	~10 dk
forks=20	~3 dk
+ persistent SSH	~2 dk
+ gather_facts=no	~1.5 dk

8.5 Dokümantasyon ve Versiyon Kontrolü

Git ile Versiyon Kontrolü

```
# Proje dizinini Git ile baslat
cd ~/network-automation
git init

# .gitignore dosyasi olustur
cat > .gitignore << 'EOF'
outputs/
*.log
*.retry
vault_pass.txt
*.pyc
__pycache__/
.ansible/
EOF

# İlk commit
git add .
git commit -m "Initial commit: Ansible ag otomasyonu projesi"

# Feature branch olustur
git checkout -b feature/yeni-vlan-ekleme

# Degisiklikleri commit et
git add playbooks/vlan-management.yml
git commit -m "feat: VLAN 400-450 araligi eklendi"
```

Playbook İçi Dokümantasyon

```
---
# ============================================================================
# Playbook: config-backup.yml
# Amaç: Tüm ağ cihazlarının yapılandırmasını yedekleme
# Yazar: Ag Ekibi
# Son Guncelleme: 2025-01-15
# ============================================================================
# Kullanim:
#   ansible-playbook -i inventory/hosts.ini playbooks/config-backup.yml
#
# Degisiklik Gecmisi:
#   2025-01-15: İlk versiyon
#   2025-01-20: Retention policy eklendi
# ============================================================================

- name: "Ağ Cihaz Yedekleme"
  hosts: all
  gather_facts: no
  ...
```

Ekler

Ek A: Hızlı Başvuru Kartı

En Sık Kullanılan Komutlar

Komut	Açıklama
`ansible -m ping all`	Tüm cihazlara ping at
`ansible -m ios_command -a "commands='show version'" cisco`	Cisco cihazlardan versiyon al
`ansible-playbook playbook.yml`	Playbook çalıştır
`ansible-playbook playbook.yml --check`	Dry-run (değişiklik yok)
`ansible-playbook playbook.yml --limit cisco`	Sadece cisco grubuna uygula
`ansible-playbook playbook.yml --tags vlan`	Sadece vlan tag'li görevleri çalıştır
`ansible-playbook playbook.yml --skip-tags backup`	backup tag'li görevleri atla
`ansible-playbook playbook.yml -v`	Detaylı çıktı
`ansible-playbook playbook.yml -vvv`	Çok detaylı çıktı
`ansible-playbook playbook.yml --step`	Her görevden önce onay sor
`ansible-doc ios_config`	Modül dokümantasyonu
`ansible-inventory --graph`	Envanter ağaç görünümü
`ansible-vault create secrets.yml`	Yeni vault dosyası oluştur

Modül Hızlı Referansı

Platform	Modül	Kullanım

Cisco IOS	`ios_command`	Show komutları çalıştırma
	`ios_config`	Yapılandırma ekleme/değiştirme
	`ios_vlan`	VLAN yönetimi
	`ios_interface`	Arayüz yönetimi
	`ios_facts`	Cihaz bilgisi toplama
	`ios_user`	Kullanıcı yönetimi
Juniper JunOS	`junos_command`	Komut çalıştırma
	`junos_config`	Yapılandırma yönetimi
	`junos_facts`	Cihaz bilgisi toplama
	`junos_system`	Sistem ayarları
	`junos_user`	Kullanıcı yönetimi
Ortak	`netconf_config`	NETCONF yapılandırma
	`netconf_get`	NETCONF veri alma

Ek B: Sık Karşılaşılan Hatalar

Hata Mesajı	Nedeni	Çözümü
`Authentication failed`	Yanlış kullanıcı adı/şifre	Inventory'de `ansible_user` ve `ansible_ssh_pass` değişkenlerini kontrol edin
`Connection timed out`	Cihaza ağ erişimi yok	IP adresini ping ile kontrol edin; güvenlik duvarı ve ACL kurallarını kontrol edin
`Unable to open shell`	SSH bağlantısı kuruldu ama shell açılamadı	Cihazda VTY hatları yapılandırılmış mı kontrol edin
`Invalid input detected`	Komut cihazda desteklenmiyor	Komutu cihazın IOS/JunOS versiyonuna göre düzeltin

`Command timeout`	Komut çok uzun sürdü	`ansible_command_timeout` değerini artırın
`Module not found`	Ansible sürümü eski	`pip install --upgrade ansible` ile güncelleyin
`Vault password not provided`	Şifreli dosyayı okumak için şifre girilmedi	`--ask-vault-pass` veya `--vault-password-file` kullanın
`undefined variable`	Değişken tanımlı değil	Değişken ismini kontrol edin; `default()` filtresi ekleyin
`Unsupported parameters`	Modül parametresi yanlış	`ansible-doc modul_adi` ile doğru parametreleri kontrol edin

Hata Ayıklama Adımları

1. **Bağlantı Testi**
   `ansible cisco -m ios_command -a "commands='show version'" -vvv` ile detaylı hata mesajı alın.

2. **Değişken Kontrolü**
   `ansible-inventory --host cihaz-adi` ile tüm değişkenleri görüntüleyin.

3. **Syntax Kontrolü**
   `ansible-playbook playbook.yml --syntax-check` ile YAML hatalarını bulun.

4. **Tek Cihaz Testi**
   `--limit cihaz-adi` parametresi ile tek bir cihazda test edin.

5. **Dry-Run Kullanımı**
   `--check` parametresi ile değişiklik yapmadan sonucu görün.

Ek C: Faydalı Kaynaklar

Ansible Resmi Kaynakları

Kaynak	URL	Açıklama
Ansible Documentation	docs.ansible.com	Resmi dokümantasyon; modül referansları
Ansible Galaxy	galaxy.ansible.com	Hazır rol ve koleksiyonlar
Ansible Network Docs	docs.ansible.com/network	Ağ otomasyonu özel dokümantasyon
Ansible GitHub	github.com/ansible/ansible	Kaynak kod ve issue takibi

Cisco Kaynakları

Kaynak	URL	Açıklama
Cisco IOS Command Reference	cisco.com/c/en/us/support/ios-nx-os-software	IOS komut referansı
Cisco DevNet	developer.cisco.com	API ve otomasyon kaynakları

Juniper Kaynakları

Kaynak	URL	Açıklama
JunOS Documentation	juniper.net/documentation	JunOS resmi dokümantasyon
Juniper TechLibrary	juniper.net/techpubs	Teknik yayınlar

Önerilen İleri Okuma

- **Ansible for DevOps** - Jeff Geerling (kitap)
- **Network Programmability and Automation** - Jason Edelman (kitap)
- **Python for Network Engineers** - Kirk Byers (ücretsiz eğitim)
- **NAPALM** - Çok satıcılı ağ otomasyon kütüphanesi; Ansible ile entegre çalışır
- **NetBox** - Ağ envanteri yönetimi; Ansible dynamic inventory kaynağı olarak kullanılabilir